QUANTIXONE

Workflow AI Builder

Backend Developer Assignment

Build All Backend Services from Scratch

Self-Contained Microservices | 9-Service Architecture | 5-Phase Delivery

Internal Assignment | All 5 Phases | March 2026

Assigned To: _____

Manager: Rajan Garg

1. Assignment Overview

Purpose

You are being assigned complete backend development of the Quantixone Workflow AI Builder. This document provides your complete scope, deliverables, acceptance criteria, and technical specifications. You will be solely responsible for designing, implementing, and deploying all nine backend microservices, their APIs, database schemas, async task processing, and infrastructure across all 5 phases.

Project Context

Quantixone is a marketing and agency platform that enables users to create and manage complex automated workflows. The Workflow AI Builder is a core feature that allows users to:

- Create automated workflows using a visual drag-and-drop interface
- Generate workflows from natural language descriptions using AI
- Trigger workflows based on CRM events, webhooks, and scheduled times
- Execute multi-step actions (SMS, email, WhatsApp, CRM updates)
- Track workflow execution metrics and performance

Your Role

As the Backend Engineer for this project, you are responsible for:

- Designing and implementing all nine backend microservices
- Building server-side APIs from scratch (NOT integrating with existing external APIs)
- Creating self-contained service modules for CRM, messaging, contacts, analytics
- Setting up async task processing with Celery and AWS SQS
- Implementing the AI/LLM pipeline for natural language workflow generation
- Managing MongoDB schemas and database operations
- Deploying infrastructure using Terraform and Docker
- Ensuring tenant isolation and multi-tenancy security
- Writing comprehensive tests and API documentation

Tech Stack

Component	Technology	Notes
Runtime	Python 3.11+	Async-first
Web Framework	FastAPI + Uvicorn	All services
Data Validation	Pydantic v2	Models & schemas
Database	MongoDB (Motor async)	Single shared cluster
Task Queue	Celery + AWS SQS	Async workflows
Cache	Redis (ElastiCache)	Session & rate limits
Containerization	Docker	One per service

Infrastructure	Terraform	IaC
Compute	AWS ECS Fargate	Serverless
LLM Providers	OpenAI + Anthropic	Dual redundancy
LLM Framework	LangChain + LangGraph	Chain composition
API Gateway	AWS API Gateway	Auth & throttling
Async Messaging	AWS EventBridge	Cross-service events

Key Principles

- Config-Driven: No hardcoded values. All configuration from environment variables and Pydantic Settings.
- Microservices-First: Each service has a single responsibility and communicates via events and APIs.
- Self-Contained Modules: Each service module owns its data and APIs end-to-end (contacts, messaging, CRM, analytics).
- Tenant Isolation: Every query must filter by tenant_id. Multi-tenancy enforced at the middleware layer.
- Type Safety: Full type hints on all functions. Pydantic models for all request/response bodies.
- Error Handling: Custom exception classes with error codes. Proper HTTP status codes and error messages.

2. Architecture Overview

Nine Backend Services

The Workflow AI Builder is composed of nine self-contained microservices. Each service owns its domain, data, and APIs. No external Quantixone services are called — you build everything from scratch.

Service	Responsibility	Technology
api-gateway	Entry point, JWT auth, tenant injection, rate limiting, API versioning	FastAPI
workflow-engine	Core workflow CRUD, DAG execution, trigger/action registries, Celery workers	FastAPI + Celery
ai-orchestrator	LLM pipelines, NL-to-workflow, refinement, template gallery, usage metering	FastAPI + LangChain
contacts-service	Contact CRUD, search, filters, tags, custom fields, bulk operations	FastAPI + MongoDB
messaging-service	SMS/Email/WhatsApp sending, templates with variables, delivery tracking, rate limiting	FastAPI + Celery
crm-service	Opportunities, pipelines, deals, pipeline stages, stage transitions	FastAPI + MongoDB
media-service	File uploads, S3 storage, media management, presigned URLs, attachments	FastAPI + S3
analytics-service	Execution metrics, daily/weekly rollups, dashboards, performance analytics	FastAPI + MongoDB
tenant-service	Multi-tenant config, subscription, billing, feature flags, quotas, branding	FastAPI

Config-Driven Architecture (4-Layer Model)

All configuration flows from environment → app settings → tenant settings → workflow settings. NO hardcoded values anywhere.

- Layer 1 (Environment): Environment variables loaded at container startup from AWS Secrets Manager
- Layer 2 (Application): Pydantic Settings singleton, validated at startup, defaults applied
- Layer 3 (Tenant): Per-tenant config in MongoDB (quotas, feature flags, channel credentials, branding)
- Layer 4 (Workflow): Per-workflow config (timeout, retry policy, channel preferences)

Pydantic Settings Example (Updated: Internal Services Only)

```
from pydantic_settings import BaseSettings
from pydantic import Field

class Settings(BaseSettings):
    # API Config
    API_VERSION: str = 'v1'
    API_HOST: str = '0.0.0.0'
    API_PORT: int = 8000

    # Database (Single MongoDB cluster)
    MONGODB_URL: str = Field(env='MONGODB_URL')
    DB_NAME: str = 'quantixone_workflows'

    # Celery / SQS
    CELERY_BROKER_URL: str = Field(env='CELERY_BROKER_URL')
    SQS_QUEUE_URL: str = Field(env='SQS_QUEUE_URL')

    # LLM
    OPENAI_API_KEY: str = Field(env='OPENAI_API_KEY')
    OPENAI_MODEL: str = 'gpt-4o'
    CLAUDE_API_KEY: str = Field(env='CLAUDE_API_KEY')

    # Auth
    JWT_SECRET: str = Field(env='JWT_SECRET')
    JWT_ALGORITHM: str = 'RS256'

    # Channel Credentials (Messaging)
    TWILIO_ACCOUNT_SID: str = Field(env='TWILIO_ACCOUNT_SID')
    TWILIO_AUTH_TOKEN: str = Field(env='TWILIO_AUTH_TOKEN')
    TWILIO_PHONE_NUMBER: str = Field(env='TWILIO_PHONE_NUMBER')
    AWS_SES_REGION: str = Field(env='AWS_SES_REGION', default='us-east-1')
    AWS_S3_BUCKET: str = Field(env='AWS_S3_BUCKET')

    # No external service URLs — we build all services!

    class Config:
        env_file = '.env'
```

Tenant Config Schema (MongoDB)

```
{
  '_id': ObjectId,
  'tenant_id': 'tenant_abc123',
  'name': 'Acme Corporation',
  'subscription_tier': 'pro',
  'branding': {
    'logo_url': 'https://...',
    'primary_color': '#1A3C5E',
    'app_name': 'Acme Automations'
  },
  'features': {
    'ai_generation': true,
    'multi_channel': true,
    'analytics': true
  },
  'quotas': {
    'max_workflows': 100,
    'max_executions_per_day': 10000,
    'max_ai_generations_per_day': 100
  },
  'channels': {
    'sms': { 'enabled': true, 'from_number': '+1...' },
    'email': { 'enabled': true, 'from_address': 'noreply@...' },
    'whatsapp': { 'enabled': true }
```

```
  },
  'created_at': ISODate,
  'updated_at': ISODate
}
```

3. Module Responsibilities (Replacing 'Existing Microservices')

This section replaces the previous section about integrating with external microservices. Now YOU build all these modules from scratch as part of the backend. Each module owns its data, APIs, and endpoints.

Module	MongoDB Collections	Key Endpoints You Build	Used By (Triggers/Actions)	Owner Service
contacts-service	contacts, contact_tags	GET/POST/PUT/DELETE /api/v1/contacts, /search, /tags, /bulk	Trigger: contact_created, Action: update_contact, add_tag	contacts-service
crm-service	opportunities, pipelines, stages	GET/POST/PUT /api/v1/opportunities, /pipelines, /stages, /move	Trigger: pipeline_stage_changed, Action: move_pipeline, create_opp	crm-service
messaging-service	messages, templates, delivery_logs	POST /api/v1/messages/sms, /email, /whatsapp, GET /templates	Action: send_sms, send_email, send_whatsapp	messaging-service
media-service	media, attachments	POST /api/v1/media/upload, GET /media/{id}, /presigned-url	Action: attach_media, email attachments	media-service
analytics-service	executions, analytics_daily, metrics	POST /api/v1/analytics/events, GET /metrics, /workflows/{id}/summary	Capture on every execution	analytics-service
workflow-engine	workflows, executions, registries	GET/POST/PUT /api/v1/workflows, /registries/triggers, /registries/actions	Core: trigger parsing, action dispatch	workflow-engine

4. Phase 1 — Foundation (Weeks 1-4)

Objective

Establish the project scaffold, configuration system, authentication, database connectivity, and health checks. This phase ensures all infrastructure is in place before building feature services. NO external service integrations needed — focus on YOUR infrastructure only.

Tasks

1. **Monorepo Scaffold**: Set up Poetry project structure with 9 services as packages. Docker images for each. Pre-commit hooks, GitHub Actions CI/CD starter config.
2. **Pydantic Settings**: Implement 4-layer config system (env → app → tenant → workflow). NO references to external service URLs. Only internal service configs (Twilio, SES, OpenAI, etc).
3. **MongoDB Connection**: Set up Motor async driver, connection pooling, indexes. Single shared MongoDB cluster. BaseRepository pattern for all queries.
4. **Collection Schemas**: Create collections: tenants, workflows, executions, contacts, opportunities, messages, media, analytics_daily, trigger_definitions, action_definitions, ai_prompt_templates.
5. **JWT Auth Middleware**: Implement RS256 token validation, token refresh logic, user context injection.
6. **RBAC Middleware**: Role-based access control (Owner/Admin/Editor/Viewer). Enforce roles on all endpoints.
7. **Tenant Isolation Middleware**: Extract and inject tenant_id on every request. Validate tenant access tokens.
8. **Health Check Endpoint**: GET /health returns service status, DB connectivity, SQS connectivity, Redis connectivity.
9. **FastAPI App Factory**: Create reusable app factory for all services. Standardize CORS, exception handlers, logging.
10. **Seed Scripts**: Create scripts to seed trigger/action registries into MongoDB. Pre-populate 20+ supported triggers and 30+ supported actions.

Deliverables Checklist

Deliverable	Acceptance Criteria	Status
Monorepo structure	✓ Poetry project with 9 services ✓ Docker images build ✓ Pre-commit hooks configured ✓ GitHub Actions on PR	☐
Config system	✓ Pydantic Settings loaded ✓ All config validated on startup ✓ No hardcoded values ✓ Tenant config in MongoDB	☐
Database setup	✓ Motor async driver works ✓ Connection pooling ✓ Indexes on all collections ✓ BaseRepository tests pass	☐
Auth middleware	✓ RS256 tokens validated ✓ Invalid tokens rejected 401 ✓ Token refresh works ✓ Integration tests pass	☐

Tenant isolation	✓ Tenant_id injected all requests ✓ Cross-tenant access blocked ✓ Audit log tracks all access ✓ Penetration test passes	☐

Deadline: End of Week 4

5. Phase 2 — Core Workflow Engine + CRM Module (Weeks 5-10)

Objective

Build the core workflow execution engine AND the first domain modules (contacts-service, crm-service). Users can create workflows, define triggers/actions, execute them asynchronously. Build your own CRM module instead of calling external APIs.

Tasks

11. **Workflow CRUD API**: POST /workflows (create), GET /workflows (list), GET /workflows/{id}, PUT /workflows/{id}, DELETE /workflows/{id}.
12. **Workflow Validation**: Validate workflow JSON schema, detect circular dependencies, enforce action/trigger limits per tenant.
13. **Trigger Registry API**: GET /registries/triggers (list all available), POST /registries/triggers/{id}/test (test webhook).
14. **Action Registry API**: GET /registries/actions (list all available), POST /registries/actions/{id}/test (test action).
15. **Celery Worker Setup**: Configure Celery with SQS broker, task auto-retry, dead-letter queue, worker monitoring.
16. **Contacts-Service (Build It!)**: POST/GET/PUT/DELETE /contacts, /contacts/search, /contacts/{id}/tags, /contacts/bulk, custom fields.
17. **Contacts Database**: Build contacts collection in MongoDB. Support search, filters, tags, bulk operations.
18. **CRM-Service (Build It!)**: POST/GET/PUT /opportunities, /opportunities/pipelines, /opportunities/{id}/move, /opportunities/{id}/stage.
19. **Opportunities Database**: Build opportunities, pipelines, stages collections in MongoDB.
20. **Workflow Execution Engine**: Implement DAG traversal algorithm. Execute workflow steps in sequence/parallel. Handle branching (if/then/else).
21. **Action Dispatcher**: Route action execution to appropriate service (contacts, messaging, crm, media). Handle retries, timeouts.
22. **Webhook Receiver**: Accept webhooks from triggers (contact_created, opportunity_stage_changed). Parse and validate events.
23. **Execution Logging & History API**: GET /workflows/{id}/executions (history), GET /workflows/{id}/executions/{exec_id} (detail with logs).
24. **Workflow Activation**: Endpoints to enable/disable workflows. Scheduled workflows stop when deactivated.

Deliverables Checklist

Deliverable	Acceptance Criteria	Status
Workflow CRUD	✓ All 5 endpoints working ✓ Validation on create/update ✓ Soft delete on DELETE ✓ Integration tests pass	☐
Contacts-Service	✓ Full CRUD implemented ✓ Search/filter working ✓ Tags system functional ✓ Bulk operations work ✓ 100 test contacts created	☐

CRM-Service	✓ Opportunities CRUD works ✓ Pipelines API functional ✓ Stage transition logic ✓ Move opportunity endpoint works ✓ Relationships preserved	☐
Execution Engine	✓ 1000 workflow steps executed ✓ DAG validation prevents loops ✓ Parallel execution works ✓ Branching logic correct ✓ Error state tracking	☐
Webhook Handler	✓ Webhooks parsed correctly ✓ Events routed to workflows ✓ Signature verification passed ✓ Load test: 100 webhooks/sec ✓ No dropped events	☐

Deadline: End of Week 10

6. Phase 3 — AI Intelligence Layer (Weeks 11-16)

Objective

Implement natural language workflow generation. Users describe workflows in plain text, and the AI generates executable workflow JSON. Include refinement API to modify workflows via NL.

Tasks

25. **AI Orchestrator Service**: FastAPI service dedicated to LLM operations. Prompt management, model routing, token usage tracking.
26. **5-Stage LLM Pipeline**: (1) Intent Classification, (2) Entity Extraction, (3) Schema Mapping, (4) Workflow Composition, (5) Confidence Scoring.
27. **Prompt Templates**: Store versioned prompt templates in MongoDB. System prompts, few-shot examples, output schemas.
28. **Dual-Provider LLM**: GPT-4o as primary (OpenAI), Claude as fallback (Anthropic). Auto-failover on rate limits or errors.
29. **Workflow Refinement API**: POST /ai/workflows/{id}/refine with natural language modification. AI modifies existing workflow.
30. **Template Gallery API**: GET /templates (browse pre-built), POST /templates/{id}/clone (copy and customize).
31. **AI Usage Metering**: Track token consumption per tenant. Store for billing. Alert on quota overages.
32. **Confidence Threshold**: Configurable per tenant. Workflows below threshold require manual approval before execution.

Deliverables Checklist

Deliverable	Acceptance Criteria	Status
LLM Pipeline	✓ All 5 stages implemented ✓ Intent classification > 95% accuracy ✓ Generated workflows validate ✓ Confidence scores calibrated	☐
NL-to-Workflow	✓ 50 test prompts → workflows ✓ Workflows syntactically valid ✓ Avg generation time < 3s ✓ Hallucination rate < 5%	☐
Dual-Provider LLM	✓ Primary provider works ✓ Fallback triggers on failure ✓ Cost tracking per provider ✓ No dropped requests	☐
Template Gallery	✓ 20+ templates available ✓ Clone operation creates copy ✓ Template categorization works ✓ Search functional	☐

Deadline: End of Week 16

7. Phase 4 — Messaging & Multi-Channel (Weeks 17-20)

Objective

Build the messaging-service from scratch. Implement SMS, email, and WhatsApp sending. Build media-service for file uploads and attachments. Implement analytics infrastructure.

Tasks

33. **Messaging-Service (Build It!)**: Implement SMS (Twilio), Email (SES), WhatsApp (Twilio). Each channel managed as separate module.
34. **Message Templates**: Store templates with variable substitution {{contact_name}}, {{amount}}. Include preview endpoints.
35. **Channel Config**: Load provider credentials from tenant config. Support multiple providers per channel.
36. **Delivery Status Tracking**: Track delivery status (pending, sent, delivered, failed). Webhook callbacks update status.
37. **Rate Limiting Per Channel**: Enforce per-tenant limits: SMS 100/min, Email 1000/day, WhatsApp 50/min. Return 429 if exceeded.
38. **Media-Service (Build It!)**: File uploads to S3, media management, presigned URLs, attachment handling.
39. **Analytics Collection**: Capture metrics on EVERY workflow execution: execution_id, workflow_id, start_time, end_time, status.
40. **Analytics Aggregation API**: POST /analytics/events, GET /analytics/workflows/{id}/metrics (daily/weekly/monthly rollups).
41. **Analytics Dashboard Data**: Return execution success rate, avg latency, error rates per workflow.

Deliverables Checklist

Deliverable	Acceptance Criteria	Status
SMS Actions	✓ SMS sent via Twilio ✓ Delivery status tracked ✓ Retries work (3x max) ✓ Rate limit enforced	☐
Email Actions	✓ Email sent via SES ✓ Templates work with vars ✓ Delivery status tracked ✓ Bounce handling	☐
Messaging-Service	✓ All three channels working ✓ Templates with variables ✓ Delivery tracking database ✓ Rate limiting enforced ✓ Integration tests pass	☐
Media-Service	✓ File uploads to S3 ✓ Presigned URLs generated ✓ Metadata tracked in MongoDB ✓ Attachments linked to messages	☐
Analytics	✓ All events captured ✓ No dropped metrics ✓ Daily/weekly rollups ✓ API latency < 500ms ✓ Accuracy verified	☐

Deadline: End of Week 20

8. Phase 5 — Production Readiness (Weeks 21-24)

Objective

Prepare for GA launch. Implement white-label support, deploy infrastructure, run load tests, conduct security audits, and establish monitoring/alerting.

Tasks

42. **White-Label API**: Support per-tenant branding (logo, colors, domain). Serve tenant-specific branding on API responses.
43. **Terraform Infrastructure**: Define ECS Fargate task definitions, load balancer, auto-scaling, SQS queues, Redis cluster, MongoDB Atlas.
44. **CI/CD Pipeline**: GitHub Actions builds Docker images, pushes to ECR, deploys to ECS. Approval gates for production.
45. **Load Testing**: Use Locust to simulate 1000 concurrent users executing workflows. Target: p95 latency < 500ms.
46. **Security Audit**: OWASP Top 10 review, penetration testing, secret scanning, SSL/TLS validation.
47. **Monitoring & Alerting**: CloudWatch dashboards for CPU, memory, request count, error rate. X-Ray distributed tracing. Alarms.
48. **API Documentation**: Generate OpenAPI/Swagger schemas. Include request/response examples. Publish to developer portal.
49. **Database Migration Scripts**: Create versioned migration scripts for schema updates. Rollback procedures. Tested on staging.
50. **Disaster Recovery**: Daily automated backups to S3. RTO < 4 hours, RPO < 1 hour. Test recovery quarterly.
51. **SDK & Integration Docs**: SDK documentation for webhook consumers. Guide for creating custom trigger/action integrations.

Deliverables Checklist

Deliverable	Acceptance Criteria	Status
Infrastructure	✓ Terraform deploys full stack ✓ Auto-scaling configured ✓ All services healthy ✓ Failover tested	☐
Load Testing	✓ 1000 concurrent workflows ✓ p95 latency < 500ms ✓ 0 errors under load ✓ CPU < 80%, memory < 70%	☐
Security Audit	✓ OWASP Top 10 passed ✓ Pen test report clean ✓ No secrets in history ✓ SSL cert validated	☐
Documentation	✓ OpenAPI spec complete ✓ 20+ endpoint examples ✓ SDK reference published ✓ Webhook docs comprehensive	☐

Deadline: End of Week 24

9. API Contracts (Endpoints You Build)

Below is a comprehensive list of ALL the key API endpoints YOU must build. All endpoints require X-Tenant-ID header and JWT Bearer token. These are YOUR APIs — not external integrations.

Authentication Endpoints

Method	Path	Request	Response	Phase
POST	/auth/login	{ email, password }	{ access_token, refresh_token }	P1
POST	/auth/refresh	{ refresh_token }	{ access_token }	P1

Workflow Endpoints (You Build All)

Method	Path	Request	Response	Phase
POST	/workflows	{ name, description, steps }	{ workflow_id, created_at }	P2
GET	/workflows	query: limit, offset, status	{ workflows: [...], total }	P2
GET	/workflows/{id}	-	{ workflow: {...} }	P2
PUT	/workflows/{id}	{ name, description, steps }	{ workflow_id, updated_at }	P2
DELETE	/workflows/{id}	-	{ success: true }	P2
POST	/workflows/{id}/activate	-	{ status: 'active' }	P2
POST	/workflows/{id}/deactivate	-	{ status: 'inactive' }	P2
POST	/workflows/{id}/execute	{ trigger_data }	{ execution_id }	P2

Registry Endpoints (Triggers & Actions)

Method	Path	Request	Response	Phase
GET	/registries/triggers	-	{ triggers: [...] }	P2
POST	/registries/triggers/{id}/test	{ data }	{ valid: bool }	P2
GET	/registries/actions	-	{ actions: [...] }	P2
POST	/registries/actions/{id}/test	{ data }	{ success: bool, result }	P2

Execution & History Endpoints

Method	Path	Request	Response	Phase

GET	/workflows/{id}/executions	query: limit, offset	{ executions: [...] }	P2
GET	/workflows/{id}/executions/{exec_id}	-	{ execution: {...} }	P2
POST	/webhooks/{trigger_type}	{ event_data }	{ received: true }	P2

Contacts Endpoints (You Build)

Method	Path	Request	Response	Phase
POST	/contacts	{ name, email, phone }	{ contact_id }	P2
GET	/contacts	query: search, limit	{ contacts: [...] }	P2
GET	/contacts/{id}	-	{ contact: {...} }	P2
PUT	/contacts/{id}	{ ...fields }	{ contact_id }	P2
DELETE	/contacts/{id}	-	{ success: true }	P2
POST	/contacts/{id}/tags	{ tags: [] }	{ success: true }	P2
POST	/contacts/bulk	{ contacts: [] }	{ created: int }	P2

Opportunities Endpoints (You Build)

Method	Path	Request	Response	Phase
POST	/opportunities	{ name, contact_id }	{ opportunity_id }	P2
GET	/opportunities	query: limit, offset	{ opportunities: [...] }	P2
GET	/opportunities/{id}	-	{ opportunity: {...} }	P2
PUT	/opportunities/{id}/move	{ stage_id }	{ success: true }	P2
GET	/opportunities/pipelines	-	{ pipelines: [...] }	P2

Messaging Endpoints (You Build - Phase 4)

Method	Path	Request	Response	Phase
POST	/messages/sms	{ to, body, template_id }	{ message_id }	P4
POST	/messages/email	{ to, subject, template_id }	{ message_id }	P4
POST	/messages/whatsapp	{ to, body, template_id }	{ message_id }	P4
GET	/messages/templates	query: channel	{ templates: [...] }	P4

GET	/messages/{id}	-	{ message: {...} }	P4

Media Endpoints (You Build - Phase 4)

Method	Path	Request	Response	Phase
POST	/media/upload	multipart/form-data	{ media_id, url }	P4
GET	/media/{id}	-	{ media: {...} }	P4
POST	/media/{id}/presigned-url	-	{ url, expires_in }	P4

AI Endpoints (You Build - Phase 3)

Method	Path	Request	Response	Phase
POST	/ai/generate	{ description }	{ workflow, confidence }	P3
POST	/ai/workflows/{id}/refine	{ instruction }	{ workflow, changes }	P3
GET	/templates	query: category, limit	{ templates: [...] }	P3
POST	/templates/{id}/clone	{ new_name }	{ workflow_id }	P3

Analytics Endpoints (You Build - Phase 4)

Method	Path	Request	Response	Phase
GET	/analytics/summary	query: period	{ total, success_rate }	P4
GET	/analytics/workflows/{id}	query: period	{ metrics: {...} }	P4
POST	/analytics/events	{ metrics }	{ recorded: true }	P4

Configuration Endpoints

Method	Path	Request	Response	Phase
GET	/config	-	{ config: {...} }	P1
GET	/tenants/me	-	{ tenant: {...} }	P1
PUT	/tenants/me	{ ...fields }	{ tenant: {...} }	P1

10. Database Schemas

All collections stored in MongoDB with tenant isolation. Every document includes tenant_id for query filtering.

workflows collection

```
{
  '_id': ObjectId,
  'tenant_id': 'tenant_abc123',
  'workflow_id': 'wf_12345',
  'name': 'Welcome Email',
  'description': 'Send welcome email to new contacts',
  'status': 'active|inactive|archived',
  'triggers': [{ 'type': 'contact_created', 'config': {} }],
  'steps': [
    { 'id': 'step_1', 'type': 'action', 'action_type': 'send_email', 'config': {...},
'next_step': 'step_2' }
  ],
  'config': { 'timeout_seconds': 300, 'max_retries': 3 },
  'created_at': ISODate, 'updated_at': ISODate
}
```

contacts collection

```
{
  '_id': ObjectId,
  'tenant_id': 'tenant_abc123',
  'contact_id': 'c_12345',
  'name': 'John Doe',
  'email': 'john@example.com',
  'phone': '+1234567890',
  'tags': ['vip', 'active'],
  'custom_fields': { 'company': 'Acme', 'industry': 'Tech' },
  'created_at': ISODate, 'updated_at': ISODate
}
```

opportunities collection

```
{
  '_id': ObjectId,
  'tenant_id': 'tenant_abc123',
  'opportunity_id': 'opp_12345',
  'name': 'Acme Deal',
  'contact_id': 'c_12345',
  'pipeline_id': 'pip_123',
  'current_stage': 'stage_2',
  'value': 50000,
  'probability': 0.75,
  'created_at': ISODate, 'updated_at': ISODate
}
```

messages collection

```
{
  '_id': ObjectId,
  'tenant_id': 'tenant_abc123',
  'message_id': 'msg_12345',
  'channel': 'sms|email|whatsapp',
  'from': 'noreply@...' | '+1234567890',
  'to': 'recipient@...' | '+1987654321',
```

```
  'body': 'Hello {{contact_name}}',
  'status': 'pending|sent|delivered|failed',
  'template_id': 'tpl_123',
  'sent_at': ISODate, 'delivered_at': ISODate,
  'created_at': ISODate
}
```

media collection

```
{
  '_id': ObjectId,
  'tenant_id': 'tenant_abc123',
  'media_id': 'media_12345',
  'file_name': 'document.pdf',
  'file_size': 1024000,
  'content_type': 'application/pdf',
  's3_url': 'https://bucket.s3.amazonaws.com/...',
  'uploaded_by': 'user_123',
  'created_at': ISODate
}
```

analytics_daily collection

```
{
  '_id': ObjectId,
  'tenant_id': 'tenant_abc123',
  'date': '2026-03-17',
  'workflow_id': 'wf_12345',
  'total_executions': 500,
  'successful_executions': 495,
  'failed_executions': 5,
  'total_actions': 1200,
  'success_rate': 0.99,
  'avg_execution_time_ms': 245,
  'p95_latency_ms': 512
}
```

11. Project Structure

Monorepo layout with all nine services and shared code. Each service is a separate FastAPI application with its own requirements.txt and Dockerfile.

```
quantixone-backend/
├── services/
│   ├── api_gateway/          # Entry point, auth, routing
│   ├── workflow_engine/      # Workflows, CRUD, execution
│   ├── ai_orchestrator/      # LLM pipelines
│   ├── contacts_service/     # Contacts, search, tags
│   ├── crm_service/          # Opportunities, pipelines
│   ├── messaging_service/    # SMS, Email, WhatsApp
│   ├── media_service/        # File uploads, S3
│   ├── analytics_service/    # Metrics, aggregations
│   └── tenant_service/       # Config, billing, quotas
├── shared/
│   ├── models/               # Pydantic models
│   ├── database/             # MongoDB client, repos
│   ├── auth/                 # JWT, RBAC, tenant middleware
│   ├── config/               # Settings
│   └── utils/                # Errors, logging
├── tests/
│   ├── unit/
│   ├── integration/
│   └── fixtures/
├── scripts/
│   ├── seed_db.py
│   └── migrate.py
├── pyproject.toml            # Poetry config
├── docker-compose.yml
└── terraform/                # Infrastructure as Code
```

12. Coding Standards & Guidelines

Code Formatting

- Use Black for code formatting (line length 100)
- Use Ruff for linting. Fix all warnings before PR.
- Pre-commit hooks enforce formatting on every commit.

Type Hints

- 100% type hints on all functions. No `Any` except where absolutely necessary.
- Use `Optional[T]` instead of `T | None`. Use typing module.
- Pydantic models for all request/response bodies. Never raw dicts.

FastAPI Patterns

- Use dependency injection for auth, db, config.
- All endpoints return Pydantic response models with proper status codes.
- Use HTTPException with proper status codes. Never raise generic Exception.

MongoDB Patterns

- All queries must include tenant_id filter. No cross-tenant data access.
- Use BaseRepository pattern for all data access. Never raw Motor queries in routes.
- Indexes created on tenant_id + query fields for performance.

Error Handling

- Custom exception classes with error codes (e.g., ERR_INVALID_WORKFLOW)
- All exceptions caught at middleware layer. Return proper JSON error responses.
- Log all errors with context (tenant_id, user_id, request_id).

Testing

- Unit tests for all business logic functions.
- Integration tests for API endpoints. Use pytest fixtures.
- Minimum 80% code coverage. Run on every PR.
- Mock external services (Twilio, SES, OpenAI) in tests.

Git Workflow

- Feature branches from main. Descriptive commit messages.
- Pull requests require code review before merge.
- Squash commits on merge. Keep history clean.

Documentation

- Docstrings on all functions. Google style.
- README for each service explaining purpose and setup.

- API endpoint docs generated from OpenAPI schema.

13. Timeline & Milestones

Phase	Duration	Key Deliverables	Sign-Off	Owner
Phase 1: Foundation	Weeks 1-4	Config system, Auth, DB setup	Rajan Garg	Dev Lead
Phase 2: Workflows + CRM	Weeks 5-10	Workflow CRUD, Contacts, Opportunities	Rajan Garg	Dev Lead
Phase 3: AI Layer	Weeks 11-16	NL-to-workflow, Templates, Refinement	Rajan Garg	Dev Lead
Phase 4: Messaging	Weeks 17-20	SMS/Email/WhatsApp, Analytics, Media	Rajan Garg	Dev Lead
Phase 5: Production	Weeks 21-24	White-label, Terraform, Load tests, Audit	Rajan Garg	Dev Lead

Weekly Standups: Every Monday 10 AM PT with Rajan Garg.

Sprint Reviews: Every 2 weeks. Demo deliverables. Adjust scope if needed.

14. Escalation & Support

Weekly Standups

Every Monday 10 AM PT. 30 minutes. Discuss progress, blockers, risks, and upcoming week's priorities.

Sprint Demos

Every 2 weeks (Fridays 2 PM PT). Demo working software. 20 min demo + 10 min Q&A.

Blocker Resolution

Report blockers immediately in #quantixone-dev Slack channel. Rajan Garg responds within 24 hours.

Resources

You have access to:

- AWS development account (credentials in 1Password)
- Twilio sandbox account for SMS/WhatsApp testing
- AWS SES for email testing
- OpenAI and Anthropic API keys in Secrets Manager
- MongoDB Atlas cluster (dev and staging)
- GitHub Enterprise for code reviews and CI/CD

Slack Channel

Join #quantixone-dev for daily communication. Async updates welcome.

15. Sign-Off

Developer Signature & Date

_____

Print Name: _____ Date: _____

Manager Signature & Date (Rajan Garg)

_____

Date: _____